Toward a Standardized Representation for Deep Learning Collective Algorithms

Jinsun Yoo and William Won, *Georgia Institute of Technology, Atlanta, GA, 30332, USA*
Meghan Cowan, Nan Jiang, Benjamin Klenk, and Srinivas Sridharan, *Nvidia, Santa Clara, CA, 95051, USA*
Tushar Krishna, *Georgia Institute of Technology, Atlanta, GA, 30332, USA*

The explosion of machine learning (ML) model size has led to its execution on distributed clusters at a large scale. Many works have tried to optimize the process of producing collective algorithms and running collective communications, which act as a bottleneck to distributed ML. However, different works use their own collective algorithm representation, resulting in a fragmented optimization environment. We propose a standardized workflow leveraging a common representation format based on Chakra Execution Trace, a widely used graph-based representation of distributed ML workloads. This common representation enables us to view collective communications at the same level as workload operations and decouple producer and consumer tools, enhancing interoperability and reducing engineering effort. We provide a proof of concept of this standardized workflow by simulating the collective algorithms generated by MSCCLang domain-specific language and the topology-aware collective algorithm synthesizer (TACOS) through the accelerator scaling for training simulator (ASTRA-sim) distributed ML simulator using multiple network configurations.

Recent trends in enormous machine learning (ML) models, such as recommendation models or large language models, have made it impractical to execute them on a single neural processing unit (NPU), such as a GPU, tensor processing unit, or custom application-specific integrated circuit.[1] Consequently, ML execution has evolved to distribute the job across multiple NPUs.[2] Within distributed ML, each of the participating NPUs completes a portion of the overall compute task. Then, they periodically transfer and synchronize their intermediate compute results (e.g., activations, weight gradients, or input gradients) in accordance with a predefined schedule.[3] These traffic patterns are collective in nature (i.e., one to many or many to one). These *collective communications*, such as All-Reduce or All-Gather, have been the key building blocks of distributed ML platforms.[4]

0272-1732 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies.
Digital Object Identifier 10.1109/MM.2025.3547363
Date of publication 5 March 2025; date of current version 29 April 2025.

The communication of intermediate data has become a bottleneck to overall distributed ML execution, and recent studies have tried to optimize this.[5,6,7] Within the scope of collectives, Collective Communication Libraries (CCLs) such as Nvidia's NCCL[8] provide implementations of several predefined *collective algorithms* (e.g., ring or double binary tree). Several works have tried to further optimize collective algorithms by designing complex algorithms,[9] automatically *synthesizing* topology-aware algorithms,[10,11] or allowing users to easily define and test their own algorithms.[12]

Unfortunately, these collective algorithm optimizations have been done separately from distributed ML workload optimization. This is largely due to a *lack of common representation* of the collective algorithm itself as well as the absence of shared representation across workload and collective communication. For example, Megatron-LM[13] or Fully Sharded Data Parallel (FSDP)[14] are mechanisms that optimize workloads by proposing new distributed training schemes. These works do not discuss how each collective communication is implemented.

Authorized licensed use limited to: University of Edinburgh. Downloaded on October 13,2025 at 01:46:58 UTC from IEEE Xplore. Restrictions apply.

Specifically, manifests describing distributed ML workload information do not contain details on how each collective algorithm is implemented and solely rely on consumer tools such as communication runtimes or simulators to fill in this blank on their own. Consumer tools, as such, have their own implementation of collective algorithms. Unfortunately, different consumer tools use their own representation to describe distinct collective algorithms, hindering interoperability and research. *Furthermore, to the best of our knowledge, there is no common representation that can capture both the distributed ML workloads and collective algorithms*. Because of this, today, collective communications and other operators within a workload (such as compute or memory access) are designed, produced, consumed, and optimized separately.

To bridge this gap, *we propose a common representation format for both workload and collective algorithm across different tools*. To achieve this, we draw our attention to the Chakra Execution Trace (Chakra ET) format, a standard graph-based representation of workload execution traces used for distributed ML.[15] In this work, we extend Chakra ET to also encode arbitrary collective algorithms. Figure 1 summarizes the idea, where 1) both the distributed ML workload and collective algorithm are represented in a common Chakra ET format and 2) downstream tools can directly ingest a single Chakra ET representation for both simulation or execution.

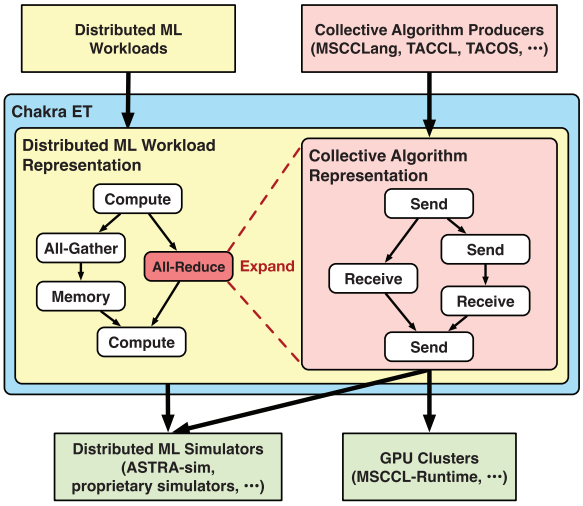

FIGURE 1. The proposed standardized workflow using Chakra ET as a common representation for both a distributed ML workload and collective algorithm. Downstream tools receive both workload and collective algorithms using the same Chakra ET format. The two graphs in the middle show a sample Chakra ET of the workload and that of the algorithm of a collective.

To showcase the benefits that this integrated representation can provide, we develop and open source a proof-of-concept workflow. We first demonstrate that different downstream tools can represent their custom collective algorithm in the standardized Chakra ET format. We leveraged MSCCLang domain-specific language (DSL)[12] and the topology-aware collective algorithm synthesizer (TACOS) collective algorithm synthesizer.[11] Then, we show that these custom collective algorithms in Chakra ET representation can be leveraged by upstream infrastructure. We simulated different collective algorithms using the accelerator scaling for training simulator (ASTRA-sim) distributed ML system simulator.[16,a] Similar workflows can be easily built by leveraging this standardized workflow.

There are three key benefits to employing Chakra ET as a common representation. First, we envision *co-optimizing collective communication with other operations in the workload via leveraging a streamlined workflow*, which encapsulates both the workload and collective algorithms.

Second, *interoperability across different tools* allows users to leverage a single representation to simulate a collective algorithm on simulators like ASTRA-sim[16] and validate the same algorithm on real systems like MSCCL-Runtime.[12]

Finally, the decoupling between tools and representation helps *reduce the effort to implement collective algorithms for each downstream tool*. For example, users can feed the common representation into ASTRA-sim without the need to implement the collective algorithm in the upstream simulator, which requires heavy simulator-specific knowledge. The standardized format enables an abstraction of both the workload and collective algorithms, allowing the research to focus on only their layer of interest.

BACKGROUND

Chakra ET

Chakra ET[15] aims to provide a standard graph-based representation that captures the trace of the distributed ML *workload* execution. Supported by the MLCommons initiative,[17] Chakra ET allows participants across both industry and academia to share workload

[a]https://github.com/astra-sim/collectiveapi.

information without revealing proprietary information. This applicability has led to its widespread adoption.

Each Chakra ET trace consists of multiple directed acyclic graphs (DAGs), where 1) vertices denote ML operations and 2) edges indicate interoperation dependencies. Each DAG represents the workload information of a single NPU, and the group of DAGs represents a distributed workload. Figure 2 shows the Chakra ET representation of a sample workload for a specific NPU. Each node in the Chakra graph holds enough information to replay the workload on hardware or simulate the workload.

Chakra ET graphs can be collected and consumed in multiple ways. For instance, users can 1) profile actual PyTorch executions over a real system or 2) synthetically generate the graphs. These graphs can be fed into and consumed by distinct downstream tools, such as distributed ML simulators or benchmarking tools, which replay the workload information on actual hardware. These downstream tools leverage Chakra ET by traversing through the graph and issuing operations, whose dependencies are resolved and ready to be dispatched.

Note that the collective communication node in Figure 2 merely indicates that an All-Gather communication has taken place but *does not elaborate on whether the actual collective algorithm is ring, double binary tree, or another*. In other words, Chakra ET itself is oblivious to exactly how the messages are orchestrated and transferred. Consequently, the downstream tools have to exploit their internal, tool-specific implementations of collective algorithms.

Upstream Collective Algorithm Producers

Given the significance of collective communication in distributed ML, numerous studies have focused on optimizing collective algorithms. Primarily, they have pursued two main threads: 1) developing DSLs to enable users to define their own collective algorithms and 2) implementing synthesizers to automatically generate collective algorithms.

One notable example of the first main thread is MSCCLang,[12] which introduces a Python-based DSL for collective algorithms. This enables users to easily construct NCCL-based collective algorithms. MSCCLang compiles these collective algorithms into an XML-based representation (MSCCL-IR), which is then executed on real clusters via the NCCL-based MSCCL-Runtime.

Meanwhile, examples of the second main thread include synthesizers like TACCL[10] and TACOS,[11] which generate collective algorithms tailored to the network topology. TACCL employs mixed-integer linear programming to identify optimal collective algorithms, while TACOS utilizes a time-expanded network (TEN) approach.

Downstream Distributed ML Tools

Downstream tools receive distributed ML workloads or collective algorithm representations to execute meaningful tasks. Collective communication runtimes and distributed ML simulators are notable instances of such downstream tools.

For instance, MSCCL-Runtime orchestrates a collective communication by taking the collective algorithm in MSCCL-IR format and executing it on real GPU clusters via NCCL-based runtime.

Conversely, ASTRA-sim[16] is a notable example of simulation infrastructure. Its workload layer receives a distributed ML workload in Chakra ET format, then parses and simulates it. Meanwhile, its system layer implements collective algorithms to fill in the gaps of the workload in Chakra ET.

REPRESENTING COLLECTIVE ALGORITHMS WITH CHAKRA ET

Motivation: Needs for Standardization

Currently, upstream collective algorithm producers employ unique representations to describe their results. For example, MSCCLang utilizes a low-level XML format, while TACOS relies on its own TEN representation.

This lack of standardization often leads downstream tools to rely on their *unique internal implementation*. Consequently, the format and pipeline that they use to fetch collective information diverge from those used to inject workload information. As a result, users are constrained to optimizing either collective operations or other workload operations, but not both. For example, existing representations are limited in that they cannot represent the idea of overlapping compute and communication, described in Wang

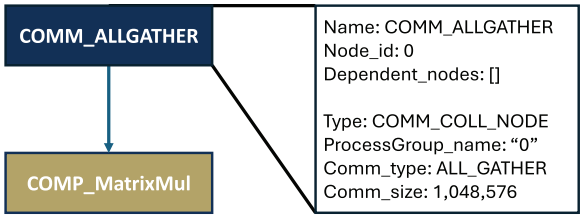

FIGURE 2. A sample depiction of the Chakra ET graph of a single NPU and the information that it contains. Note that the word *node* in the fields refers to the node of the DAG, not the physical machine.

Authorized licensed use limited to: University of Edinburgh. Downloaded on October 13,2025 at 01:46:58 UTC from IEEE Xplore. Restrictions apply.

et al.[18] With Chakra ET as-is, we can only represent workloads where the NPU waits for the All-Gather to fully complete before executing the matrix multiplication.

Moreover, the absence of a standard format means that executing an upstream producer's algorithm with a specific downstream tool requires users to comprehend the internal details of both tools and implement the algorithm themselves. This task is not only highly prohibitive but also implies that it must be repeated for every pair of upstream and downstream tools. Consequently, upstream and downstream processes become disjointed and lack plug-and-play functionality.

Solution: Using Chakra ET

We standardize the collective algorithm representation by utilizing the Chakra ET format that is already employed for distributed ML workloads. The Chakra ET schema readily offers mechanisms that capture point-to-point message transfers between arbitrary NPUs as well as compute operations necessary for collectives.

By representing both distributed ML workloads and collective algorithms in Chakra ET, we cleanly resolve the issue of separation between workloads, upstream, and downstream tools. Figure 1 depicts the proposed standardized workflow. Both distributed ML workloads and collective algorithms are in Chakra ET format and passed on to the downstream. The downstream tool then traverses the workload ET and executes operations as their dependencies are resolved. During the process, because collective communication nodes lack the exact mechanism to execute collectives, the downstream tools must decide the algorithm. Previously, they selected a collective algorithm from a range of native implementations or custom algorithms tailored to their specifications. However, with the capability to easily receive any collective algorithm in Chakra ET, the downstream tools can expand the collective communication node with the provided algorithm. Users can rapidly test new collective algorithms by simply switching out the Chakra ET files without creating new tool-specific implementations.

Additionally, by representing collective algorithms with the same format as the workload, we elevate the send and receive messages of a collective algorithm to the same level as other operators in the workload. This opens up opportunities for co-optimizing collective communication and other operators such as compute. We discuss a specific example toward the end of the "Example: Fine-Grained Compute-Communicate Overlap".

Description: Representing Arbitrary Collective Algorithms

We provide a detailed example of how we represent collective communication algorithms using Chakra ET. Figure 3(a) depicts a readily available workload trace consisting of compute and Reduce-Scatter communications. A Reduce-Scatter communication results in the participating NPUs reducing their data in place (using computations such as summations). Although there are different algorithms for achieving this end result, we look at the ring algorithm.

Figure 3(b) and (d) collectively illustrate a ring-based Reduce-Scatter across four NPUs. In the ring algorithm, participating NPUs form a logical ring. For example, NPU1 repeatedly sends messages to NPU2 and receives messages from NPU4. Figure 3(b) depicts the first message. Each NPU sends a chunk of data to the next NPU and receives another chunk of data at a different position from the previous NPU. Once the incoming message arrives, the NPU reduces the incoming chunk with the local chunk at the same position.

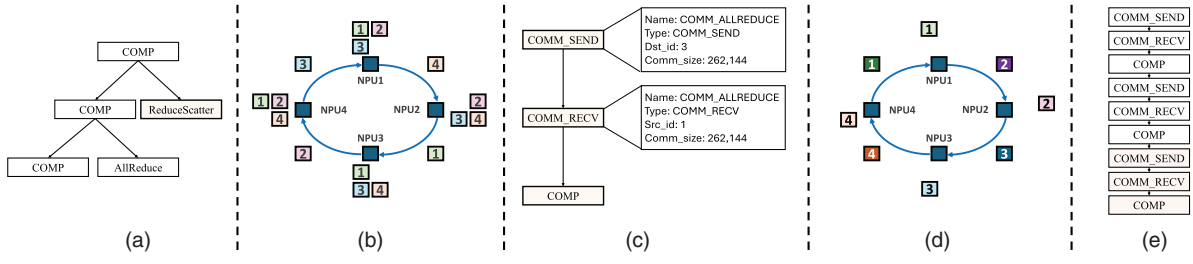

FIGURE 3. A ring-based Reduce-Scatter communication with Chakra ET. (a) A sample workload of a single NPU represented with Chakra ET. The highlighted Reduce-Scatter node is expanded with our Chakra ET communication representation. (b) Depiction of the first message. Each box is a message chunk marked with the chunk index. (c) Chakra ET graph for the message. Note that the source and destination NPU ID is unique to each NPU. (d) Depiction of the last message. The darker color indicates the chunks that have been reduced. (e) Chakra ET graph for the whole communication. The highlighted nodes indicate the last message.

Authorized licensed use limited to: University of Edinburgh. Downloaded on October 13,2025 at 01:46:58 UTC from IEEE Xplore. Restrictions apply.

This is repeated until each NPU has one fully reduced chunk, each at different positions [Figure 3(d)].

As discussed, we can break down collective communications into a point-to-point network transfer between two NPUs and compute operations. Table 1 lists the types of readily available Chakra ET operator nodes that are used to represent collective algorithms and their definition. Leveraging these nodes, Chakra ET represents arbitrary collective algorithms as a combination of network message transfers and reductions.

Figure 3(c) depicts the first message of an NPU with a sequence of COMM_SEND, COMM_RECV, and COMP Chakra ET nodes. The graph is extended with the same Chakra ET nodes. Because each DAG is unique to an NPU, we encode both the sender and receiver NPU ID in the send and receive nodes, respectively. This information allows us to capture the interaction and communication dependencies *between* NPUs, and represent topology-aware algorithms. Note how the resulting graph is effectively a single line with sequential operators. This is due to the inherent simplicity of the ring algorithm. Advanced algorithms or communications that span multiple dimensions can result in graphs with complex dependencies. We provide examples of capturing advanced, arbitrary algorithms in the evaluation.

To meet the standard workflow, upstream tools need to convert resulting collective algorithms from their default representation to the common Chakra ET format when producing collective algorithms. We highlight that implementing this conversion is a one-time task such that, once developed, can be reused across multiple downstream frameworks.

Example: Fine-Grained Compute-Communicate Overlap

One example of co-optimization that we can capture with our approach is overlapping compute and communication in a fine-grained manner. Consider the workload depicted in Figure 2, which is similar to the example provided in Wang et al.[18] Without overlapping, we would have to wait for the All-Gather to fully complete before starting the matrix multiplication. However, we can start multiplying locally available rows and columns independently with the collective chunks. Because the workload representation is isolated from collective algorithm representation, it is difficult to express such optimizations with the existing Chakra ET schema as-is.

Figure 4 shows an example of how we can represent the idea of overlapping the compute and communication with our standardized representation, as discussed earlier in this section. We first expand the All-Gather into multiple send and receive Chakra ET nodes. We then reorder the Chakra ET nodes to allow simultaneous execution of compute and communication. Chakra ET nodes without connecting edges can be executed in a nonblocking manner. For example, the NPU can start executing the first compute operation asynchronously while exchanging messages, which are captured with the first send and receive nodes.

Our representation makes it easier to explore different overlap granularities. For example, users could employ various degrees of grouping messages, which would result in different numbers of compute operations. Another possible direction is to test a scheduling feature that reorders a compute node and a send node (assuming that the reorder respects internode dependencies) as the compute and send node now use a common Chakra ET format.

Note that the relevance of our common representation is not limited to collective-workload co-optimization. This representation can also be used specifically for collective optimization and evaluation, where downstream tools do not necessarily take workload information as input. For example, a user may want to simulate multiple prospective algorithms using ASTRA-sim, then validate the best-performing candidates by actually using real-world systems such as MSCCL-Runtime. Even when MSCCL-Runtime does

TABLE 1. A list of Chakra ET node types used to represent collective algorithms and their description.

Chakra ET Node Type	Description
COMM_SEND	Send a point-to-point message to a destination
COMM_RECV	Wait for a point-to-point message that a source will send
COMP	Run a compute task (e.g., reduction)

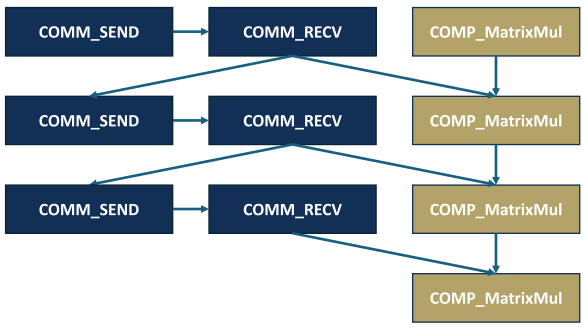

FIGURE 4. An example of representing overlapping compute and communication with our representation.

Authorized licensed use limited to: University of Edinburgh. Downloaded on October 13,2025 at 01:46:58 UTC from IEEE Xplore. Restrictions apply.

not take workload information as input, the user in this scenario can reuse the same Chakra ET format across both tools without modification or engineering effort. This is achieved because the common representation abstracts away the details of the downstream tools.

PROOF-OF-CONCEPT METHODOLOGY

We provide proof-of-concept case studies to examine our common representation's strength in allowing users to freely explore collective communication optimization by connecting upstream and downstream tools. To do so, we construct an end-to-end workflow that compares collective algorithm designs written with MSCCLang or generated with TACOS using the ASTRA-sim simulator. Here, we describe the extensions made to the tools for the case study. *We expect that similar extensions would apply to other upstream and downstream tools*.

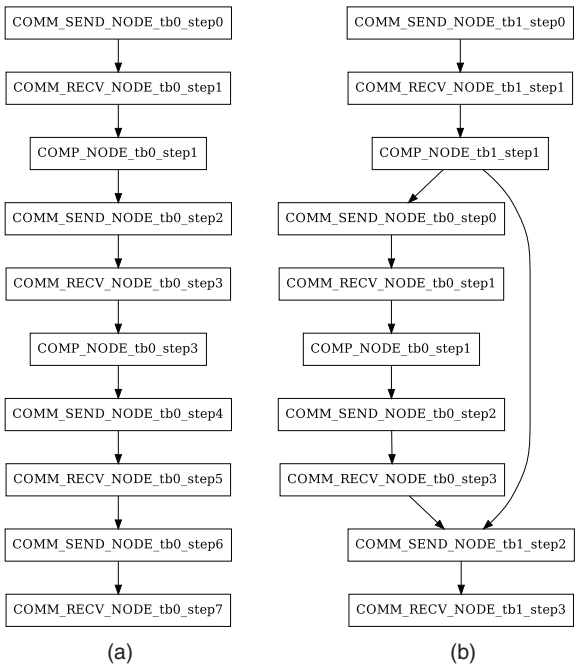

FIGURE 5. A standardized Chakra ET representation of (a) a three-NPU ring and (b) four-NPU recursive halving-doubling All-Reduce algorithm. Only the nodes of NPU 0 (as opposed to NPU 1, NPU 2, ..., NPU n) are visualized. We retrieve these representations by converting the MSCCL-IR representation generated by MSCCLang.

TABLE 2. The duration of generating Chakra ET traces from MSCCLang for All-Reduce.

Number of NPUs	16	32	64	128
Duration (ms)	259	398	1485	7662

Representing MSCCL-IR in Chakra ET

First, we developed a converter that bridges the MSCCL-IR collective algorithm representation into Chakra ET. The converter creates a vertex (i.e., Chakra ET node) for each operation in the MSCCL-IR XML representation. The converter then creates edges by extracting interoperator dependency information encoded in the MSCCL-IR representation.

TO DO SO, WE CONSTRUCT AN END-TO-END WORKFLOW THAT COMPARES COLLECTIVE ALGORITHM DESIGNS WRITTEN WITH MSCCLang OR GENERATED WITH TACOS USING THE ASTRA-sim SIMULATOR.

To showcase the update, we described a 1-D ring algorithm of All-Reduce using MSCCLang and compiled the result in standard Chakra ET format. Figure 5 demonstrates the ring and recursive halving-doubling All-Reduce algorithms[9] presented in Chakra ET representation. These graphs are obtained by first representing the corresponding collective algorithms in MSCCLang DSL, compiling them into the MSCCL-IR representation, and using the converter to standardize them into Chakra ET. Table 2 shows the end-to-end duration of generating MSCCL-IR XML representation and converting it to Chakra ET graphs for All-Reduce collectives of a varying number of NPUs.

Representing TACOS Output in Chakra ET

We also modified the TACOS collective algorithm synthesizer to generate Chakra ET as its synthesis output, rather than its custom TEN-based representation. Similar to representing MSCCL-IR output in Chakra ET, each chunk communicating over a TEN link is represented as two Chakra ET nodes: a COMM_SEND node in the sender NPU, and a COMM_RECV node in the receiver NPU. Note that when an NPU sends a chunk to one of its neighboring NPUs, it must have already received it from another NPU. Our implementation checks this interoperation dependency from the TEN and marks it as edges in the resulting Chakra ET graph. TACOS takes 1080 ms to synthesize

Authorized licensed use limited to: University of Edinburgh. Downloaded on October 13,2025 at 01:46:58 UTC from IEEE Xplore. Restrictions apply.

an All-Reduce communication for 128 NPUs (for a detailed discussion on synthesis duration, please refer to Shoeybi et al.)[11]

Figure 6 depicts an example Chakra ET representation of the All-Gather collective algorithm, synthesized by TACOS. It can be seen that each COMM_SEND operation depends on a corresponding COMM_RECV node, which first delivers the chunk into the NPU.

Updating ASTRA-sim to Run Collective Algorithms in Chakra ET

Figure 7 shows the original workflow of the ASTRA-sim distributed ML simulator and our extensions. ASTRA-sim includes its implementation of collective algorithms found in NCCL, such as ring or recursive halving-doubling, out of the box. At each run, the user chooses which collective algorithm to use for each collective communication. Whenever a collective communication node is issued, ASTRA-sim runs the corresponding algorithm.

We extend ASTRA-sim by adding an input parameter that accepts the Chakra ET representation of the collective algorithm. The simulator parses the provided Chakra ET and simulates the point-to-point send and receive communications while following all the dependencies, rather than using its internal collective algorithm implementation. This process is depicted in the dashed boxes in Figure 7. As ASTRA-sim readily supports the execution of ML workloads in Chakra ET, reusing the components in the workload layer has made it easy to extend for the common collective algorithm representations.

EVALUATION

We showcase our case study of bridging upstream collective producers and ASTRA-sim, as described in the "Proof-of-Concept Methodology" section. We run

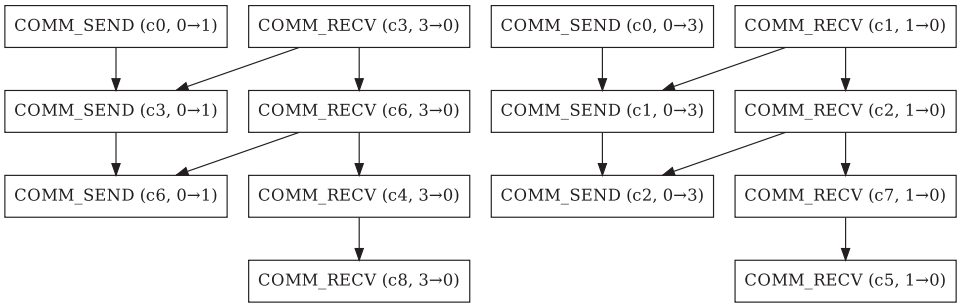

FIGURE 6. An example All-Gather algorithm synthesized by TACOS, represented in the standardized Chakra ET format. Only the nodes of NPU 0 (as opposed to NPU 1, NPU 2, ..., NPU n) are visualized.

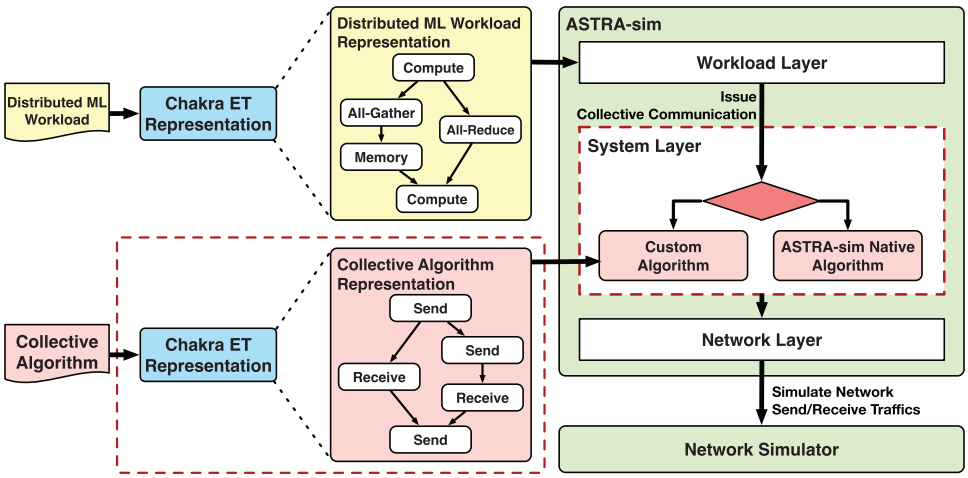

FIGURE 7. The components of ASTRA-sim involved in the case study. We extended ASTRA-sim to inject collective algorithms represented in Chakra ET. The extension is marked with dashed squares.

Authorized licensed use limited to: University of Edinburgh. Downloaded on October 13,2025 at 01:46:58 UTC from IEEE Xplore. Restrictions apply.

evaluations on a 2-D mesh and 3-D hypercube topology. For an All-Gather communication node, we used MSCCLang to generate both a ring and direct algorithm, and TACOS to generate an optimized collective tailored to the mesh and hypercube topology. Figure 3(c) and (e) shows snippets of the Chakra ET represented as a graph. The resulting Chakra ET is then provided to the ASTRA-sim simulator to be tested across different physical topologies.

We use ASTRA-sim's analytical network simulator[11] to model the message transfer. The topologies consist of 64 NPUs. We observe the collective communication bandwidth as we differ in the collective size. Figure 8 shows the bandwidth of the different collective algorithms. We were able to streamline the workflow by standardizing the collective algorithm representation, which enabled the experiment without much engineering effort.

Note that the purpose of this evaluation is to showcase a workflow using Chakra ET-based representation to compare algorithms from different upstream producers. In this scenario, it is natural that the topology-aware algorithms produced by TACOS perform better than the simple ring and direct algorithms. As the MSCCLang DSL is a tool for developers to easily write custom algorithms, evaluations with different custom algorithms could show different results. Although this evaluation showcases a simplistic workload, it is possible to expand into complex workloads and collective algorithms. We leave this to future work.

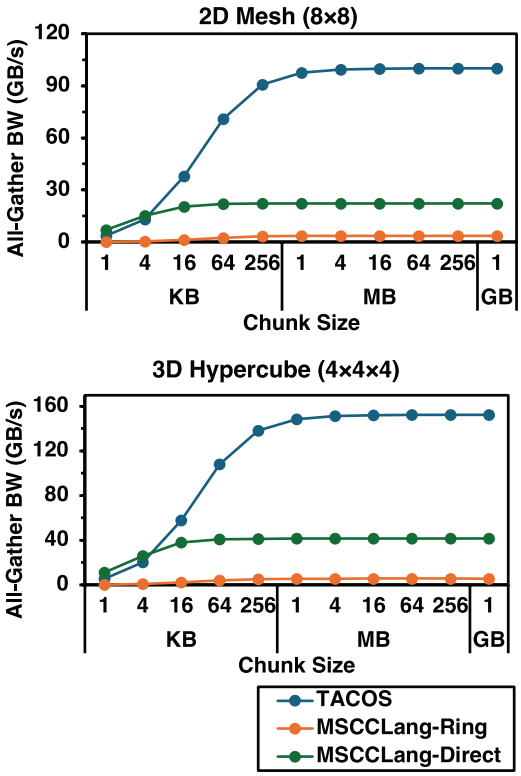

FIGURE 8. The collective bandwidth for an All-Gather communication across different upstream algorithms (MSCCLang DSL and TACOS synthesizer) and topologies (2-D mesh and 3-D hypercube) of 64 NPUs. Each link latency is 500 ns, and the bandwidth is 50 GB/s. BW: bandwidth.

CONCLUSION AND FUTURE WORK

This article proposed a standardized common representation for collective algorithms. We suggest leveraging the Chakra ET format, which already captures distributed ML workload traces, to also represent collective algorithms. Representing both workloads and collective algorithms with the same format will allow us to explore the co-optimization of workload operators and collective communication operators. Additionally, using a common collective algorithm representation across tools allows us to easily study collectives using different tools in a plug-and-play manner. We showcased the end-to-end workflow of capturing arbitrary, complex collective algorithms with a case study using collective algorithms produced by MSCCLang and TACOS on the ASTRA-sim simulator.

Note that our proposition is focused on standardizing representations. Tasks such as design space exploration for efficient collective algorithms or parallelization strategy are left to other works. Our work simply provides a common representation that opens room for co-optimization and interoperability.

Our work opens up several future research directions. One important future work is to leverage the proposed standard representation to explore collective communication optimizations in the context of an actual ML workload. We anticipate that a common representation across workloads and collective algorithms will allow us to study co-optimization, such as overlapping compute and communication operations. Another potential direction would be to enable other tools to produce and consume Chakra ET format, further expanding the ecosystem and scope of our proposed workflow. Finally, we could extend the Chakra ET format to capture hardware-based collective communication support such as Nvidia SHARP[19] and study its impact on distributed training.

Authorized licensed use limited to: University of Edinburgh. Downloaded on October 13,2025 at 01:46:58 UTC from IEEE Xplore. Restrictions apply.

ACKNOWLEDGMENTS

Jinsun Yoo and William Won are co-first authors.

REFERENCES

1. Y. Huang et al., "GPipe: Efficient training of giant neural networks using pipeline parallelism," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2019, pp. 103–112.
2. G. Wang et al., "ZeRO++: Extremely efficient collective communication for giant model training," in *Proc. 12th Int. Conf. Learn. Representations (ICLR)*, 2024. [Online]. Available: https://openreview.net/forum?id=gx2BT0a9MQ
3. Z. Tang, S. Shi, W. Wang, B. Li, and X. Chu, "Communication-efficient distributed deep learning: A comprehensive survey," 2023, *arXiv:2003.06307*.
4. B. Klenk, N. Jiang, G. Thorson, and L. Dennison, "An in-network architecture for accelerating shared-memory multiprocessor collectives," in *Proc. 47th IEEE/ACM Annu. Int. Symp. Comput. Archit. (ISCA)*, 2020, pp. 996–1009, doi: 10.1109/ISCA45697.2020.00085.
5. S. Rashidi et al., "Enabling compute-communication overlap in distributed deep learning training platforms," in *Proc. 48th IEEE/ACM Annu. Int. Symp. Comput. Architect. (ISCA)*, 2021, pp. 540–553, doi: 10.1109/ISCA52012.2021.00049.
6. A. Sapio et al., "Scaling distributed machine learning with in-network aggregation," in *Proc. 18th USENIX Symp. Netw. Syst. Des. Implementation (NSDI)*, 2021, pp. 785–808.
7. W. Won, S. Rashidi, S. Srinivasan, and T. Krishna, "LIBRA: Enabling workload-aware multi-dimensional network topology optimization for distributed training of large AI models," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, 2024, pp. 205–216, doi: 10.1109/ISPASS61541.2024.00028.
8. "NVIDIA collective communications library (NCCL)." NVIDIA Developer. Accessed: Jan. 13, 2025. [Online]. Available: https://developer.nvidia.com/nccl
9. R. Thakur, R. Rabenseifner, and W. Gropp, "Optimization of collective communication operations in MPICH," *Int. J. High Perform. Comput. Appl.*, vol. 19, no. 1, pp. 49–66, 2005, doi: 10.1177/1094342005051521.
10. A. Shah et al., "TACCL: Guiding collective algorithm synthesis using communication sketches," in *Proc. 20th USENIX Symp. Netw. Syst. Des. Implementation (NSDI)*, 2023, pp. 593–612.
11. W. Won, M. Elavazhagan, S. Srinivasan, S. Gupta, and T. Krishna, "TACOS: Topology-aware collective algorithm synthesizer for distributed machine learning," in *Proc. 57th IEEE/ACM Int. Symp. Microarchit. (MICRO)*, 2024, pp. 856–870, doi: 10.1109/MICRO61859.2024.00068.
12. M. Cowan, S. Maleki, M. Musuvathi, O. Saarikivi, and Y. Xiong, "MSCCLang: Microsoft collective communication language," in *Proc. 28th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst. (ASPLOS)*, 2023, pp. 502–514.
13. M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training multi-billion parameter language models using model parallelism," in 2020, *arXiv:1909.08053*.
14. Y. Zhao et al., "PyTorch FSDP: Experiences on scaling fully sharded data parallel," in *Proc. VLDB Endowment*, 2023, vol. 16, no. 12, pp. 3848–3860, doi: 10.14778/3611540.3611569.
15. S. Sridharan et al., "Chakra: Advancing performance benchmarking and co-design using standardized execution traces," 2023, *arXiv:2305.14516*.
16. W. Won, T. Heo, S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, "ASTRA-sim2.0: Modeling hierarchical networks and disaggregated systems for large-model training at scale," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw. (ISPASS)*, 2023, pp. 283–294, doi: 10.1109/ISPASS57527.2023.00035.
17. "Chakra." MLCommons. Accessed: Jan. 13, 2025. [Online]. Available: https://mlcommons.org/working-groups/research/chakra
18. S. Wang et al., "Overlap communication with dependent computation via decomposition in large deep learning models," in *Proc. 28th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst. (ASPLOS)*, 2022, pp. 93–106.
19. R. L. Graham et al., "Scalable hierarchical aggregation and reduction protocol (SHARP)™: Streaming-aggregation hardware design and evaluation," in *Proc. 35th Int. Conf. High Perform. Comput. (ISC)*, 2020, pp. 41–59.

JINSUN YOO is a Ph.D. student at the Georgia Institute of Technology, Atlanta, GA, 30332, USA. His research interests include distributed machine learning, collective communications, and edge computing. Yoo received his B.S. degree in computer science and engineering from Seoul National University. He is the co-corresponding author of this article. Contact him at jinsun@gatech.edu.

WILLIAM WON is a Ph.D. candidate at the Georgia Institute of Technology, Atlanta, GA, 30332, USA. His research interests include software–hardware optimizations for distributed machine learning (ML), collective communication optimizations, and simulation infrastructures for large-scale ML systems. Won

Authorized licensed use limited to: University of Edinburgh. Downloaded on October 13,2025 at 01:46:58 UTC from IEEE Xplore. Restrictions apply.

received his M.S. in computer science from Georgia Institute of Technology. He is a Graduate Student Member of IEEE. He is the co-corresponding author of this article. Contact him at william.won@gatech.edu.

MEGHAN COWAN is a software engineer at Nvidia, Santa Clara, CA, 95051, USA. Her research interests include compilers, program synthesis, and machine learning systems. Cowan received her Ph.D. degree in computer science from the University of Washington. Contact her at mcowan@nvidia.com.

NAN JIANG is a principal research scientist at Nvidia, Santa Clara, CA, 95051, USA. His research interests include switch architectures, congestion control algorithms, and networks on chip. Jiang received his Ph.D. degree in electrical engineering from Stanford University. Contact him at tedj@nvidia.com.

BENJAMIN KLENK is a senior research scientist at Nvidia, Santa Clara, CA, 95051, USA. His research interests include accelerated interconnection networks, systems for high-performance computing/artificial intelligence, and networks on chip. Klenk received his Ph.D. degree in computer engineering from Heidelberg University. Contact him at bklenk@nvidia.com.

SRINIVAS SRIDHARAN is a distinguished engineer at Nvidia, Santa Clara, CA, 95051, USA. His research interests include large-scale artificial intelligence system design, workload optimization, and production deployment. Sridharan received his doctorate in computer science and engineering from the University of Notre Dame. He is a Member of IEEE and the Association for Computing Machinery. Contact him at srisridharan@nvidia.com.

TUSHAR KRISHNA is an associate professor of electrical and computer engineering/computer science at the Georgia Institute of Technology, Atlanta, GA, 30332, USA. His research interests include computer architectures, interconnection networks, and artificial intelligence systems. Krishna received his Ph.D. degree in electrical engineering and computer science from the Massachusetts Institute of Technology. He is a Senior Member of IEEE. Contact him at tushar@ece.gatech.edu.

Authorized licensed use limited to: University of Edinburgh. Downloaded on October 13,2025 at 01:46:58 UTC from IEEE Xplore. Restrictions apply.